



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

DevOps-CA2 Assignment

MediTwin AI — End-to-End DevOps Implementation

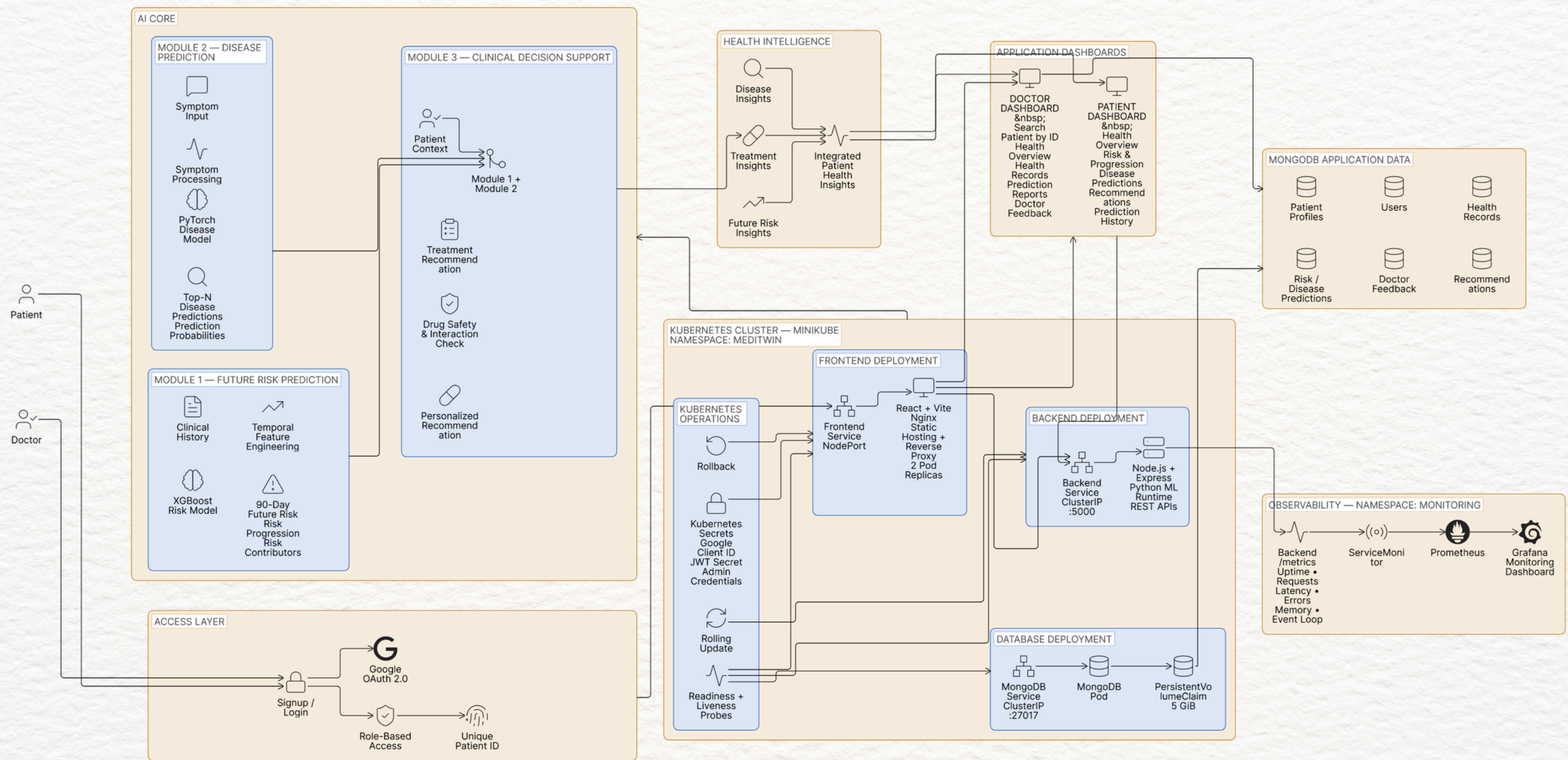
CI/CD • Configuration Management • Containerization • Kubernetes • Observability

Group Number: Group – 19

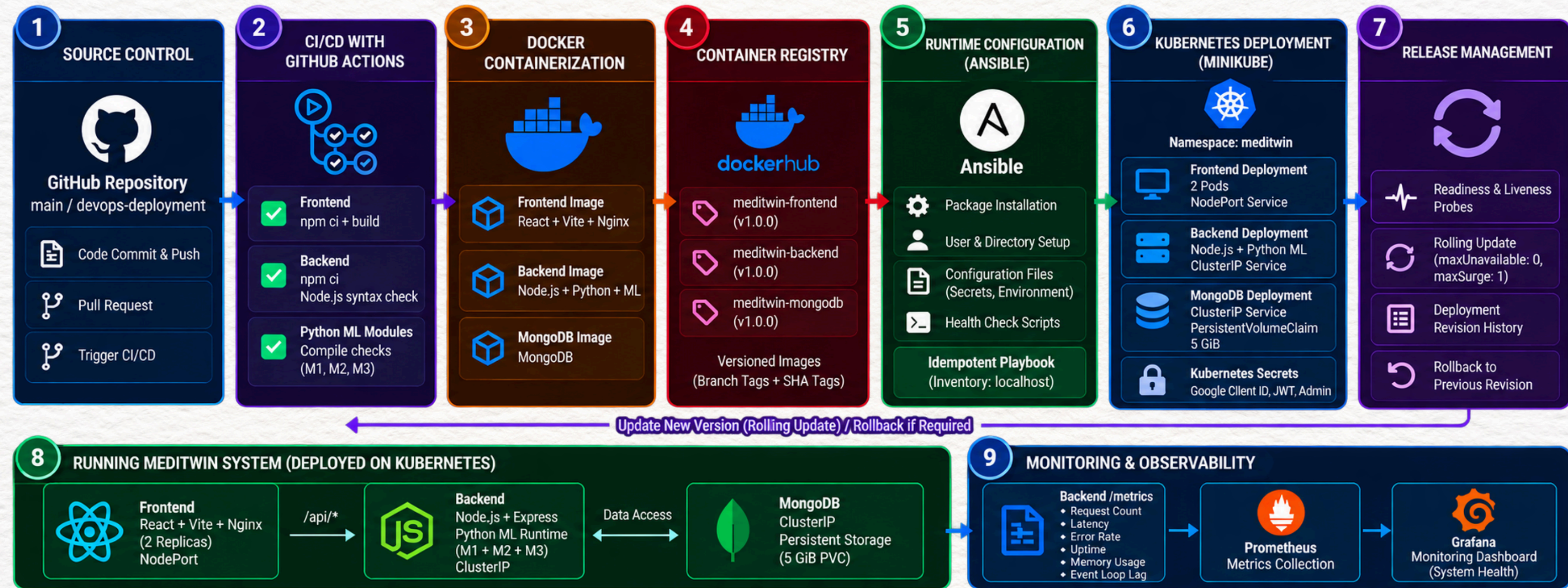
Adarsh Jha – 23070122261
Jyoti Kumari Sah- 23070122278
Muskan Shah – 23070122279
Prabin Yadav – 23070122280

Under
Arijit Dutta

System Architecture



Pipeline Flow



Automated validation → containerization → configuration → deployment → release → monitoring

Deployment & Configuration

☐	meditwin	-	-	-	1.18%	270.09MB / 22.8:	3.47%	313.6M				
☐	mongodb	d5a45eeef668	mongo:8	27017:27017	0.94%	175.1MB / 7.61G	2.25%	163MB				
☐	frontend	4fff9be7f150	meditwin-f	8080:80	0%	15.39MB / 7.61G	0.2%	10.6MB				
☐	backend	bc43143c349e	meditwin-b	5000:5000	0.24%	79.6MB / 7.61GB	1.02%	140MB				

Figure 1 – Dockerized MediTwin Services

```
PS C:\Users\yadav\Desktop\SEM 7\B-Tech Project\MediTwin> docker tag meditwin-frontend:latest meditwin-fro
PS C:\Users\yadav\Desktop\SEM 7\B-Tech Project\MediTwin> minikube image load meditwin-frontend:v2
PS C:\Users\yadav\Desktop\SEM 7\B-Tech Project\MediTwin> kubectl set image deployment/frontend frontend=m
meditwin
deployment.apps/frontend image updated
PS C:\Users\yadav\Desktop\SEM 7\B-Tech Project\MediTwin> kubectl rollout status deployment/frontend -n me
Waiting for deployment "frontend" rollout to finish: 1 out of 2 new replicas have been updated...
Waiting for deployment "frontend" rollout to finish: 1 out of 2 new replicas have been updated...
Waiting for deployment "frontend" rollout to finish: 1 out of 2 new replicas have been updated...
Waiting for deployment "frontend" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "frontend" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "frontend" rollout to finish: 1 old replicas are pending termination...
deployment "frontend" successfully rolled out
PS C:\Users\yadav\Desktop\SEM 7\B-Tech Project\MediTwin> kubectl get pods -n meditwin
NAME                                READY   STATUS    RESTARTS   AGE
backend-77d5dbf746-tmtrb            1/1     Running   0           13m
frontend-847d5dfff6-7hvb9          1/1     Running   0           16s
frontend-847d5dfff6-lmwp6          1/1     Running   0           27s
mongodb-56d8c94dbf-ph88s           1/1     Running   0           33h
PS C:\Users\yadav\Desktop\SEM 7\B-Tech Project\MediTwin> kubectl rollout history deployment/frontend -n m
deployment.apps/frontend
REVISION  CHANGE-CAUSE
1         <none>
2         <none>
3         <none>
4         <none>
```

Figure 2 – Kubernetes Deployment, Rolling Update & Rollout Verification

```
yadav@prabin:/mnt/c/Users/yadav/Desktop/SEM 7/B-Tech Project/MediTwin/ansible$
uid=1001(meditwin) gid=1002(meditwin) groups=1002(meditwin)
yadav@prabin:/mnt/c/Users/yadav/Desktop/SEM 7/B-Tech Project/MediTwin/ansible$
total 24
drwxr-xr-x 4 meditwin meditwin 4096 Oct  2 09:01 .
drwxr-xr-x 3 root      root      4096 Oct  2 09:01 ..
-rw-r--r-- 1 meditwin meditwin  199 Oct  2 09:01 DEPLOYMENT_INFO.txt
drwxr-xr-x 2 meditwin meditwin 4096 Oct  2 09:01 config
-rwxr-xr-x 1 meditwin meditwin  383 Oct  2 09:01 health_check.sh
drwxr-xr-x 2 meditwin meditwin 4096 Oct  2 09:01 logs
yadav@prabin:/mnt/c/Users/yadav/Desktop/SEM 7/B-Tech Project/MediTwin/ansible$
APP_NAME=MediTwin
APP_ENV=production
FRONTEND_PORT=8080
BACKEND_PORT=5000
PYTHON_EXEC=python
DEPLOYMENT_PLATFORM=container
yadav@prabin:/mnt/c/Users/yadav/Desktop/SEM 7/B-Tech Project/MediTwin/ansible$
MediTwin deployment environment
Checking required commands...
Python3: OK
Git: OK
Curl: OK
Environment check completed.
```

Figure 3 – Ansible Runtime Configuration & Environment Validation

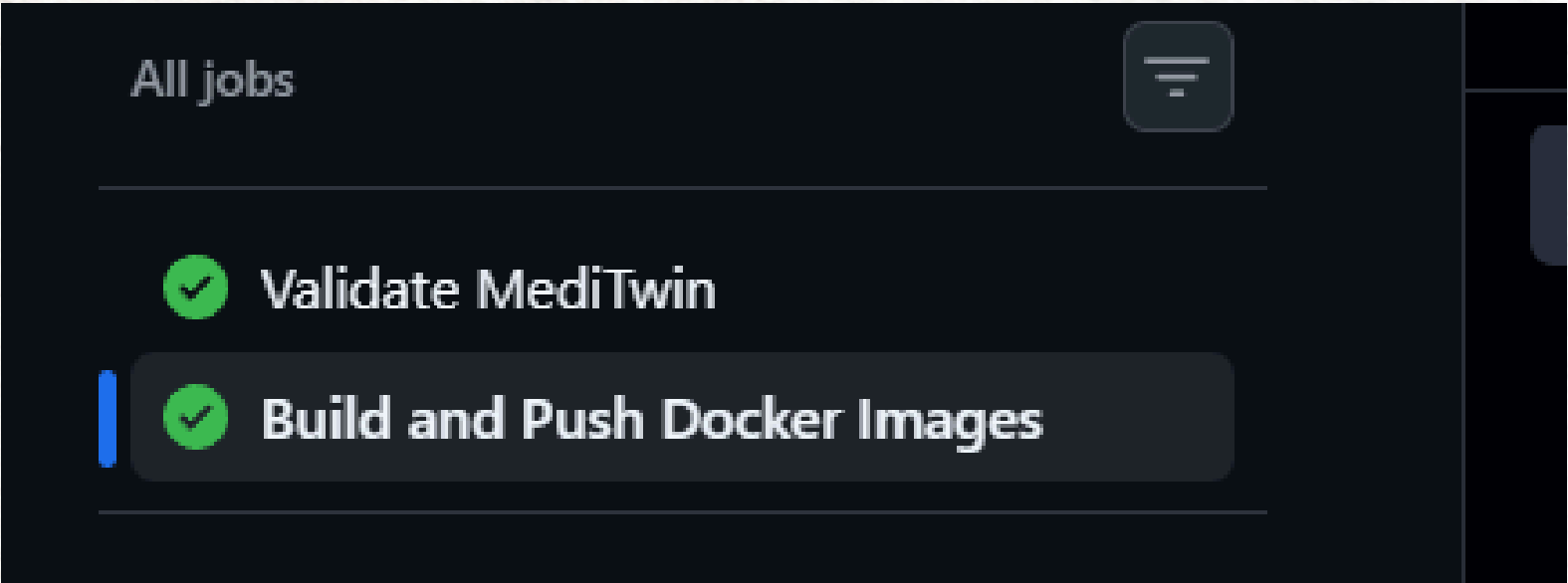


Figure 4 – GitHub Actions CI/CD Execution

Monitoring & Observability

Why Monitor?

- ✓ **Availability** — detect service downtime
- ✓ **Performance** — track latency and traffic
- ✓ **Runtime Health** — observe memory and event-loop behavior
- ✓ **Reliability** — identify HTTP errors

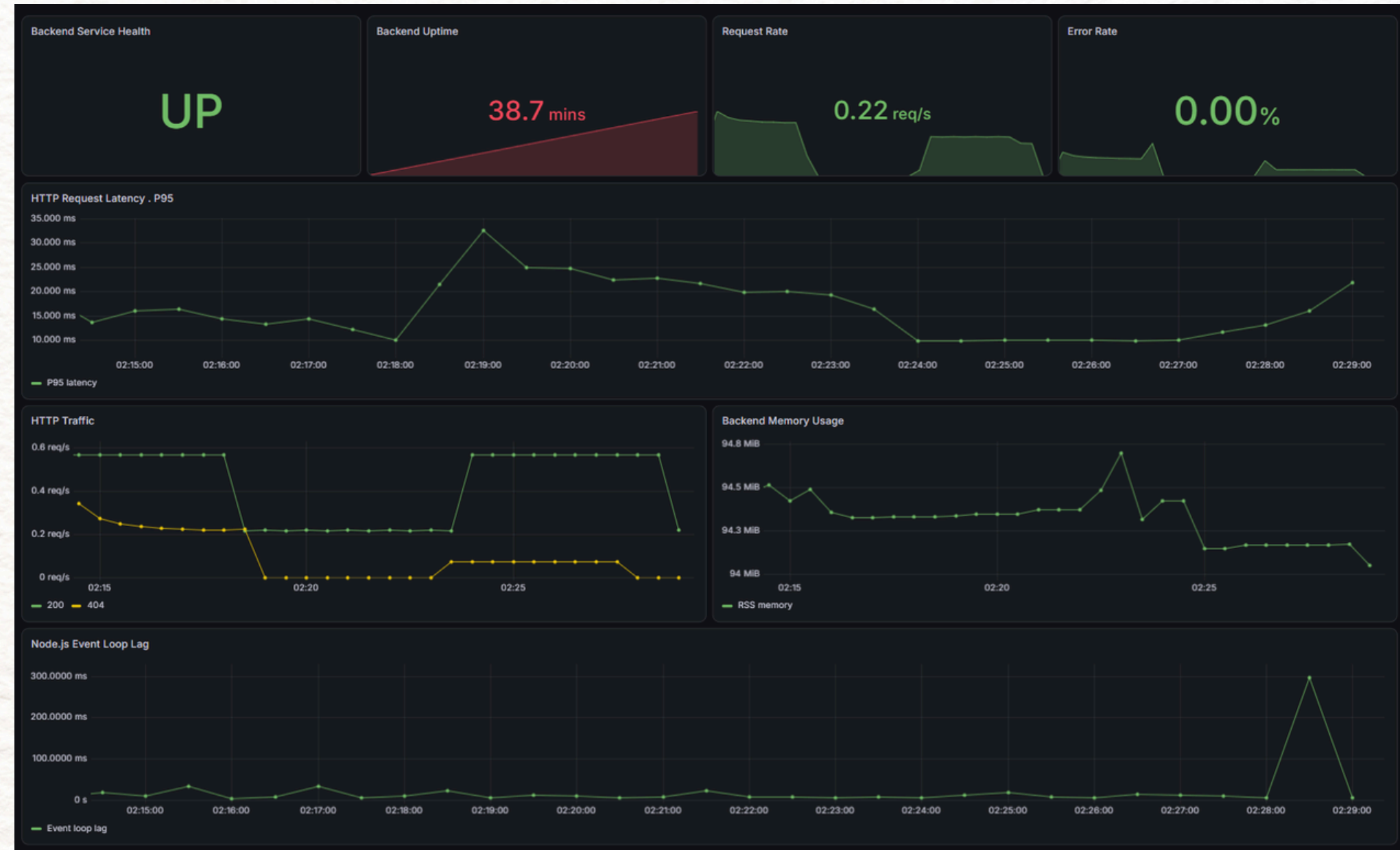


Figure 5 — MediTwin DevOps Monitoring Dashboard in Grafana

**PROMETHEUS +
GRAFANA**

/metrics → ServiceMonitor
→ Prometheus → Grafana

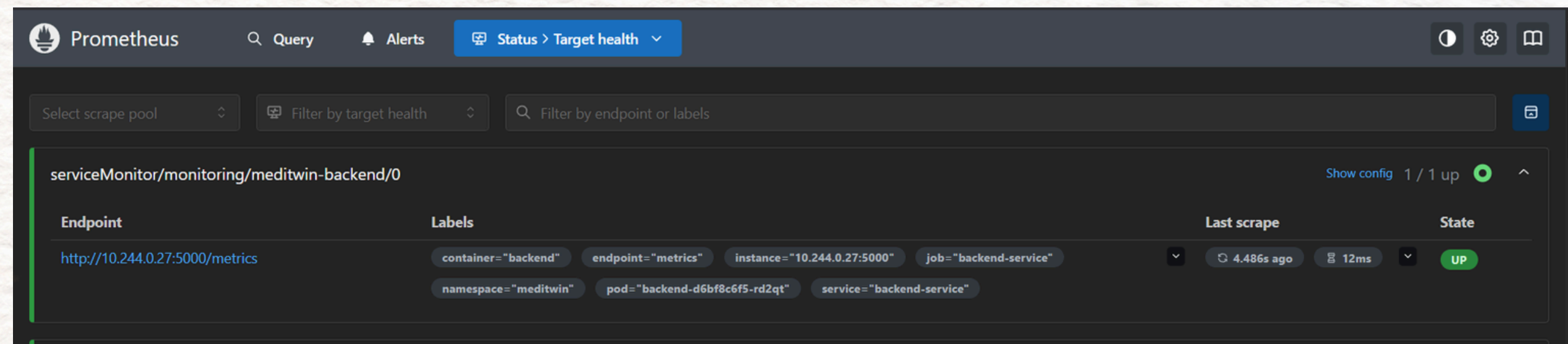


Figure 6 — MediTwin backend successfully discovered and scraped by Prometheus

Challenges & Lessons Learned



Large ML Docker Image
PyTorch made the backend image very large.



Kubernetes Runtime Issues
Startup and readiness needed careful configuration.



Google OAuth origin_mismatch
The deployed origin initially wasn't authorized.

Challenges



Reused layers and optimized the build approach.



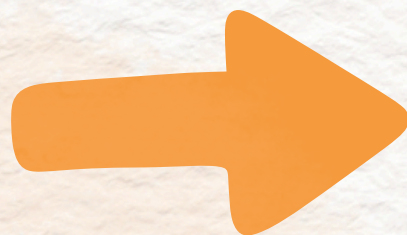
Configured Services, probes and persistent storage.



Updated the client ID and authorized origin.

Solutions

LESSONS LEARNED



Dockerized services make deployment consistent across environments.



Ansible reduces manual runtime setup and supports idempotent execution.



Kubernetes manages services, replicas, storage, health and deployment state.



Application-level metrics are essential for understanding real system behaviour.